

* *Работата с паметта е ваша отговорност. Очаква се реализация на канонично представяне във всички класове, където това е **необходимо**. Операциите да НЕ позволяват два обекта да поделят обща памет.*

* *Решенията, които не спазват ООП парадигмата, показаните добри практики и не се компилират се оценяват с 0м.*

* **Задължително** е решението на задачата да бъде придружено от `main` функция, която тества реализираната функционалност.

* *За решението на задачите трябва да се използват посочените типове. Позволено е използването на `std::vector` и `std::string` за всички места, за които **НЕ** е изрично указан тип.*

* `MAX_ARR_SIZE` е глобална константа. Да се приеме, че **НЯМА** да се добавят елементи над този размер.

1.0) **(0.5т.)** Да се реализира клас човек (Person), който се представя чрез:

- *уникален идентификатор (id) - цяло положително число, което автоматично нараства с 1*
- *име (name) - низ с максимална дължина 200 символа (позволено е използване и на `std::string`)*
- *възраст (age) - цяло положително число*
- *визитна картичка (businessCard) - низ с максимална дължина 500 (позволено е използване и на `std::string`)*
- *умения (skills) - реално число в затворения интервал [0, 1]*

В класът да се дефинира конструктор с четири параметъра и да **НЯМА** такъв без параметри.

1.1) **(0.6 т.)** Да се реализира клас за телевизионен формат (TVFormat). Класът се представя чрез:

- *участници (participants) - масив с максимална дължина `MAX_ARR_SIZE` от подходящ тип (позволено е използване и на `std::vector`)*
- *водител (hostOfTheShow) - човек, който изпълнява дадената роля*
- *събития (events) - масив от `MAX_ARR_SIZE` събития. Всяко събитие се представя с динамичен низ от тип `char*` (**НЕ** е позволено използване на `std::string` и `std::vector`).*

(0.25 т.) Да се реализира конструктор с параметри, който да инициализира данните.

(0.4 т.) От класа **НЕ** трябва да е възможно създаването на обекти и трябва да разполага със следния интерфейс:

- *`void showEvents(<подходящ тип> limit)` - извежда на екрана последните `limit` събития, които са се случили. Ако няма толкова събития, да се изведат всички събития и да се изведе съобщение, че няма повече налични събития.*
- *`void printFormatInformation()` - отпечата цялата информация за водещия и всички участници.*
- *`void doEvent()` - без конкретна реализация за класа*

2.1) **(0.5 т.)** Да се реализира клас FMChallenge. FMChallenge е телевизионен формат с водещ и точно 14 участници. В телевизионния формат има списък от **точно** 10 игри, които се играят **последователно**. Всяка игра се представя чрез низ от тип `char *` с **динамична дължина**.

(0.5 т.) Да се предефинира метод `doEvent`, който изиграва първата игра в списъка от игри, премахва я от него, намира и премахва участника, който напуска формата и добавя събитие в масива от събития (events) - *"Game was #game_description #participant_name was eliminated"*, където `game_description` е описанието на играта, `participant_name` е името на участника, който напуска формата.

При всяка игра напуска този участник, за когото съотношението умения/възраст е най-лошо.

3.1) **(0.25 т.)** Да се реализира клас KitchenFormat. KitchenFormat е телевизионен формат с водещ и точно 12 първоначални участници. В телевизионния формат се поддържат два отбора, които могат да имат **максимум** 6 участници. Да се реализира конструктор с параметри и валидация, че в отборите участват само действителни участници.

**Жокер: За реализацията на отбор можете да използвате вектор или масив от `ids` на участниците (не е позволено използване на `std::pair`).*

(0.5 т.) Да се предефинира метод `doEvent`, който взима участника в отбора с повече хора, който има най-малко умения, премахва го от отбора и добавя събитие в масива от събития (events) - *"#participant_name was eliminated"*.

Ако броят на хората в двата отбора съвпадат, да се вземе участник от първия отбор.

4.1) **(0.5 т.)** Да се реализира **шаблон** на клас KitchenBulgaria, който е разширение на стандартния формат, но има допълнителна характеристика скрито предизвикателство (*hiddenChallenge*) от **произволен** тип. Да се реализира конструктор с параметри и да се предефинира `printFormatInformation`, така че да извежда цялата информация, вкл. и *hiddenChallenge*. *Приема се, че операторът << е предефиниран за типа.*

4.2) **(0.5 т.)** Да се реализира главна функция с динамичен масив от **различни** телевизионни формати. В масива да има поне 1 формат от всеки тип. Да се изпълнят събитията `doEvent` за всеки един формат и да се отпечата новата им информация.

* *Работата с паметта е ваша отговорност. Очаква се реализация на канонично представяне във всички класове, където това е **необходимо**. Операциите да НЕ позволяват два обекта да поделят обща памет.*

* *Решенията, които не спазват ООП парадигмата, показаните добри практики и не се компилират се оценяват с 0м.*

* **Задължително** е решението на задачата да бъде придружено от `main` функция, която тества реализираната функционалност.

* *За решението на задачите трябва да се използват посочените типове. Позволено е използването на `std::vector` и `string` за всички места, за които **НЕ** е изрично указан тип.*

* `MAX_ARR_SIZE` е глобална константа. Да се приеме, че **НЯМА** да се добавят елементи над този размер.

1.0) **(0.5т.)** Да се реализира клас човек (Person), който се представя чрез:

- *уникален идентификатор (id) - цяло положително число, което автоматично нараства с 1*
- *име (name) - низ с максимална дължина 200 символа (позволено е използване и на `std::string`)*
- *възраст (age) - цяло положително число*
- *визитна картичка (businessCard) - низ с максимална дължина 500 (позволено е използване и на `std::string`)*
- *интелект (intellect) - реално число в затворения интервал [0, 1]*

В класът да се дефинира конструктор с четири параметъра и да няма такъв без параметри.

1.1) **(0.6 т.)** Да се реализира клас за телевизионна игра (TVGame). Класът се представя чрез:

- *участници (participants) - масив с максимална дължина `MAX_ARR_SIZE` от подходящ тип (позволено е използване и на `std::vector`)*
- *водител (hostOfTheGame) - човек, който изпълнява дадената роля*
- *събития (events) - масив от `MAX_ARR_SIZE` събития. Всяко събитие се представя с динамичен низ от тип `char*` (**НЕ** е позволено използване на `std::string` и `std::vector`).*

(0.25 т.) Да се реализира конструктор с параметри, който да инициализира данните.

(0.4 т.) От класа НЕ трябва да е възможно създаването на обекти и трябва да разполага със следния интерфейс:

- *`void showEvents(<подходящ тип> limit)` - извежда на екрана последните `limit` събития, които са се случили. Ако няма толкова събития, да се изведат всички събития и да се изведе съобщение, че няма повече налични събития.*
- *`void printFormatInformation()` - отпечатава цялата информация за водещия и всички участници.*
- *`void doEvent()` - без конкретна реализация за класа*

2.1) **(0.5 т.)** Да се реализира клас `WomenGame`. `WomenGame` е телевизионна игра с водещ и точно 11 първоначални участници. В телевизионния формат има списък от **точно** 12 задачи, които се случват **последователно**. Всяка задача се представя чрез низ от тип `char *` с **динамична дължина**.

(0.5 т.) Да се предефинира метод `doEvent`, който проиграва първата задача, премахва я от списъка със задачи, намира и премахва участника, който напуска играта и добавя събитие в масива от събития (events) - *"Task was #task_description #participant_name was left"*, където `task_description` е описанието на задача, `participant_name` е името на момичето, което напуска телевизионната игра. При всяка задача напуска тази участничка, за която съотношението интелект/възраст, е най-лошо.

3.1) **(0.25 т.)** Да се реализира клас `CoupleGame`. `CoupleGame` е телевизионна игра с водещ, точно 12 първоначални участници. Участниците са разделени в 6 двойки. Да се реализира конструктор с параметри и валидация, че в двойките участват само действителни участници.

**Жокер: За реализация на двойките можете да използвате вектор или масив от `ids` на участници (не е позволено използване на `std::pair`)*

(0.5 т.) Да се предефинира метод `doEvent`, който взима двойката, която сумарно има най-малко умения, премахва я от отбора и добавя събитие в масива от събития (events) - *"#participant_name1 and #participant_name2 were eliminated"*.

4.1) **(0.5 т.)** Да се реализира **шаблон** на клас `CoupleBulgariaGame`, който е разширение на стандартната игра, но има допълнителна характеристика скрито предимство (`hiddenAdvantage`) от **произволен** тип. Да се реализира конструктор с параметри и да се предефинира `printFormatInformation`, така че да извежда цялата информация, вкл. и `hiddenAdvantage`. *Приема се, че операторът `<<` е предефиниран за типа.*

4.2) **(0.5 т.)** Да се реализира главна функция с динамичен масив от **различни** телевизионни игри. В масива да има поне 1 игра от всеки тип. Да се изпълнят събитията `doEvent` за всяка игра и да се отпечата новата им информация.